

SQL Final Round Interview Questions

Senior Level - Advanced SQL Concepts

1. Window Functions vs GROUP BY

Question:

"When would you use window functions instead of GROUP BY?"

What They Test:

Advanced SQL knowledge, analytics understanding, performance awareness.

Your Ideal Approach:

Explain the difference with a practical example.

Sample Answer:

Window functions keep all rows while adding running totals, ranks, or offsets (ROW_NUMBER(), RANK(), LAG(), LEAD()). GROUP BY aggregates and reduces rows to one per group. Use window functions for analytics that need detail rows preserved, like showing each employee's salary with department average. Use GROUP BY when you only need aggregate results. Window functions are better for reporting dashboards.

2. CTE vs Subquery Performance

Question:

"CTE vs Subquery - which is faster and why?"

What They Test:

Query optimization knowledge, understanding of query execution.

Your Ideal Approach:

Compare both with execution plan consideration.

Sample Answer:

Performance depends on the query optimizer's execution plan. CTEs are more readable and can be referenced multiple times (sometimes materialized). Correlated subqueries execute per row making them slower. Non-correlated subqueries run once. Always use EXPLAIN PLAN or execution statistics to check actual performance. In most modern databases, well-written CTEs and subqueries perform similarly, but CTEs are preferred for readability and reusability.

3. Indexing Strategy for Large Tables

Question:

"How do you design indexes for a 1TB fact table?"

What They Test:

Database design, performance tuning, production experience.

Your Ideal Approach:

Show systematic indexing strategy.

Sample Answer:

First, create composite indexes on frequently used WHERE and JOIN columns (like date, customer_id). Use covering indexes that include SELECT columns to avoid table lookups. Avoid indexing low-cardinality columns (gender, status). Monitor index usage with system views like sys.dm_db_index_usage_stats to remove unused indexes. Update index statistics regularly. Test index impact before applying to production.

4. Pivoting Data Without PIVOT Function

Question:

"Write a query to pivot sales data by month without PIVOT function."

What They Test:

SQL creativity, CASE statement usage, data transformation.

Your Ideal Approach:

Use CASE with GROUP BY instead.

Sample Answer:

```
SELECT customer_id,  
       SUM(CASE WHEN month='Jan' THEN sales END) as Jan,  
       SUM(CASE WHEN month='Feb' THEN sales END) as Feb  
FROM sales GROUP BY customer_id;
```

This approach works across all SQL databases unlike PIVOT which is database-specific. It's more flexible for adding/removing months.

5. Recursive CTE for Hierarchy

Question:

"How to query employee-manager hierarchy?"

What They Test:

Understanding of recursive queries, complex data retrieval.

Your Ideal Approach:

Build recursive CTE step by step.

Sample Answer:

```
WITH hierarchy AS (  
    SELECT emp_id, name, manager_id, 1 as level  
    FROM employees WHERE manager_id IS NULL  
    UNION ALL  
    SELECT e.emp_id, e.name, e.manager_id, h.level+1  
    FROM employees e JOIN hierarchy h ON e.manager_id = h.emp_id  
)  
SELECT * FROM hierarchy;
```

This traverses the hierarchy from top to bottom, showing each employee's level from the CEO.

6. MERGE Statement Use Case

Question:

"When would you use MERGE instead of INSERT/UPDATE?"

What They Test:

Advanced DML usage, data warehouse patterns, production scenarios.

Your Ideal Approach:

Explain upsert pattern with business example.

Sample Answer:

MERGE handles both INSERT and UPDATE in a single statement (upsert operation). Perfect for data warehouse loads where you need to insert new customers and update existing ones. MERGE ensures single transaction consistency - all changes succeed or all fail. Good for Slowly Changing Dimension (SCD) Type 2 implementations. Atomic operation means no partial updates.

7. Query Plan Analysis

Question:

"How do you identify slow query bottlenecks?"

What They Test:

Troubleshooting skills, production experience, optimization mindset.

Your Ideal Approach:

Show systematic debugging approach.

Sample Answer:

Use EXPLAIN or EXPLAIN ANALYZE to view execution plans. Look for sequential scans on large tables (should be index scans), high-cost operations, missing indexes. Check if statistics are outdated with UPDATE STATISTICS. Watch for parameter sniffing where queries run differently with different inputs. Use system views like sys.dm_exec_query_stats to find most resource-intensive queries. Test with ACTUAL execution plan to see real runtime stats.

8. Handling NULLs in Aggregations

Question:

"How to handle NULLs in GROUP BY and aggregations?"

What They Test:

NULL handling expertise, data quality awareness.

Your Ideal Approach:

Use functions and CASE for NULL management.

Sample Answer:

Use COALESCE() or ISNULL() to replace NULLs with defaults. For grouping, NULLs group together by default but you can use CASE to create meaningful labels:

```
SELECT COALESCE(dept, 'Unknown') as dept_name, COUNT(*)  
FROM employees GROUP BY COALESCE(dept, 'Unknown');
```

This ensures NULLs don't create mystery rows and are counted consistently.

9. Partitioning Large Tables

Question:

"When and how to partition a 100GB table?"

What They Test:

Database architecture, scalability knowledge, big data handling.

Your Ideal Approach:

Explain partitioning strategy with business benefits.